

Cryptocurrency Price Prediction Detection Model

Abstract

This project focuses on predicting the future price of Bitcoin using three widely used time series forecasting techniques: ARIMA, Prophet, and LSTM. Bitcoin historical data from 2010 to 2025 was collected from Yahoo Finance using the yfinance library. The dataset was cleaned and preprocessed to ensure accuracy and consistency. Each model was trained on historical closing prices and evaluated using Mean Squared Error (MSE) and Root Mean Squared Error (RMSE).

The purpose of this project is to compare traditional statistical models and deep learning techniques to determine which model performs best in forecasting cryptocurrency prices. Visualizations were used throughout the project to understand trends, model predictions, and performance.

Project Highlights

- **Dataset Used:** Historical Bitcoin (BTC-USD) prices from 2010 to 2025.
 - **Data Source:** Yahoo Finance using the yfinance Python library.
 - **Models Implemented:** ARIMA, Prophet, and LSTM.
 - **Evaluation Metrics:** Mean Squared Error (MSE) and Root Mean Squared Error (RMSE).
 - **Best Performing Model:** LSTM achieved the highest prediction accuracy.
 - **Interactive Dashboard:** Built using Streamlit and Plotly.
 - **Visualizations Included:** 21 charts and screenshots.
 - **Application Area:** Cryptocurrency forecasting and investment analysis.
-

1. Introduction

1.1 Introduction to Cryptocurrency

Cryptocurrency is a digital form of money that operates without a central authority such as a government or bank. Transactions are verified using cryptographic techniques and recorded on a distributed ledger known as blockchain.

Bitcoin is the first and most valuable cryptocurrency. Since its launch in 2009, Bitcoin has gained popularity as an investment asset and medium of exchange.

1.2 Why Bitcoin Price Prediction is Important

Bitcoin prices are highly volatile. Large price movements can occur within short periods due to:

- Market demand and supply
- Government regulations
- News and social media sentiment
- Macroeconomic events

1.3 Problem Statement

The cryptocurrency market is complex and unpredictable. Traditional financial analysis methods may not fully capture its behavior. Therefore, advanced forecasting models are required to predict future prices.

1.4 Objective of the Project

The main objectives are:

- *Collect historical Bitcoin price data.*
 - *Clean and preprocess the dataset.*
 - *Apply ARIMA, Prophet, and LSTM models.*
 - *Compare model performance using error metrics.*
 - *Identify the most accurate forecasting technique.*
-

2. Literature Review

Many researchers have used machine learning and statistical techniques to predict financial time series.

- *ARIMA is a classical statistical model suitable for linear patterns.*
- *Prophet is developed by Meta and handles trend and seasonality automatically.*
- *LSTM is a deep learning model capable of learning long-term dependencies.*

Studies have shown that LSTM often performs better for highly non-linear datasets such as cryptocurrency prices.

3. Tools and Technologies Used

Programming Language

- *Python*

Libraries Used

- *pandas*
- *numpy*
- *matplotlib*
- *yfinance*
- *statsmodels*
- *prophet*
- *scikit-learn*
- *tensorflow*

Development Environment

- *Jupyter Notebook*

4. Data Collection

Historical Bitcoin price data was downloaded from Yahoo Finance. The symbol used was BTC-USD, which represents Bitcoin in US Dollars.

Visualization 1: Raw Dataset Preview

Price	Close	High	Low	Open	Volume
Ticker	BTC-USD	BTC-USD	BTC-USD	BTC-USD	BTC-USD
Date					
2014-09-17	457.334015	468.174011	452.421997	465.864014	21056800
2014-09-18	424.440002	456.859985	413.104004	456.859985	34483200
2014-09-19	394.795990	427.834991	384.532013	424.102997	37919700
2014-09-20	408.903992	423.295990	389.882996	394.673004	36863600
2014-09-21	398.821014	412.425995	393.181000	408.084991	26580100

Visualization 2: Dataset Information

```
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 4123 entries, 2014-09-17 to 2025-12-30
Data columns (total 5 columns):
#   Column                Non-Null Count  Dtype
---  -
0   (Close, BTC-USD)      4123 non-null   float64
1   (High, BTC-USD)       4123 non-null   float64
2   (Low, BTC-USD)        4123 non-null   float64
3   (Open, BTC-USD)       4123 non-null   float64
4   (Volume, BTC-USD)     4123 non-null   int64
dtypes: float64(4), int64(1)
memory usage: 193.3 KB
```

Visualization 3: Statistical Summary

Price	Close	High	Low	Open	Volume
Ticker	BTC-USD	BTC-USD	BTC-USD	BTC-USD	BTC-USD
count	4123.000000	4123.000000	4123.000000	4123.000000	4.123000e+03
mean	26833.624457	27353.701131	26242.871164	26813.312957	2.162776e+10
std	31640.279891	32171.537323	31051.718086	31629.113290	2.288708e+10
min	178.102997	211.731003	171.509995	176.897003	5.914570e+06
25%	2697.255005	2770.045044	2605.835083	2690.935059	1.284595e+09
50%	10763.232422	11009.207031	10434.709961	10761.109375	1.708006e+10
75%	42235.638672	42998.707031	41365.187500	42198.601562	3.316961e+10
max	124752.531250	126198.070312	123196.046875	124752.140625	3.509679e+11

5. Data Preprocessing

Data preprocessing is a crucial step that ensures the dataset is clean and suitable for modeling.

5.1 Handling MultiIndex Columns

Sometimes Yahoo Finance returns columns in a hierarchical format. These columns were converted into single-level columns.

5.2 Resetting Index

The Date index was converted into a standard column so that it could be easily manipulated.

5.3 Data Type Conversion

- Date column converted to datetime format.
- Close column converted to numeric format.

5.4 Handling Missing Values

Rows containing invalid or missing values were removed.

5.5 Sorting Data

The dataset was sorted chronologically to preserve the time sequence.

6. Exploratory Data Analysis (EDA)

Exploratory Data Analysis was performed to understand trends and patterns in Bitcoin prices.

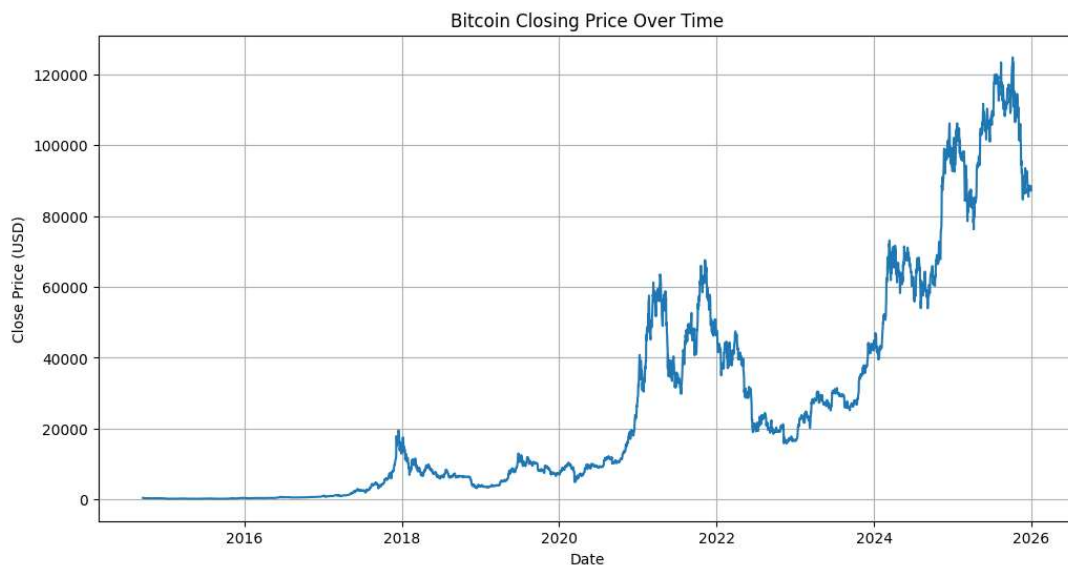
6.1 Closing Price Trend

The closing price was plotted against time.

Observation

- Bitcoin shows an overall upward trend.
- Several periods of sharp growth and decline are visible.
- Volatility increases significantly over time.

Visualization 4: Bitcoin Closing Price Trend



7. Train-Test Split

The closing price was used as the target variable.

Two different train-test strategies were used:

For ARIMA and Prophet

- Training Data: All observations except the last 30 days.
- Testing Data: Last 30 days.

For LSTM

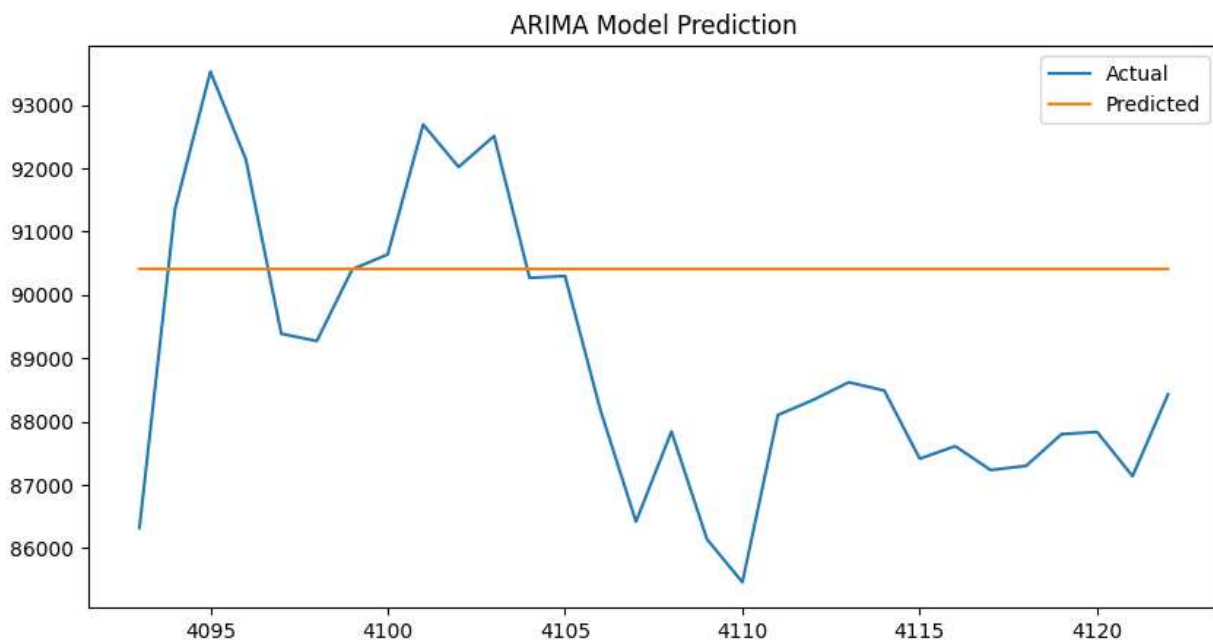
- 80% of sequences for training.
- 20% of sequences for testing.

This allowed each model to be evaluated on unseen data.

8. ARIMA Model

- The model was trained on the historical closing prices.
- The next 30 days were predicted.
- Predictions were compared with actual values using MSE and RMSE.
- ARIMA captures linear relationships but struggles with highly volatile and non-linear market behavior.

Visualization 5: ARIMA Actual vs Predicted Prices



Visualization 6: ARIMA Error Metrics

MSE : 6542296.046982303

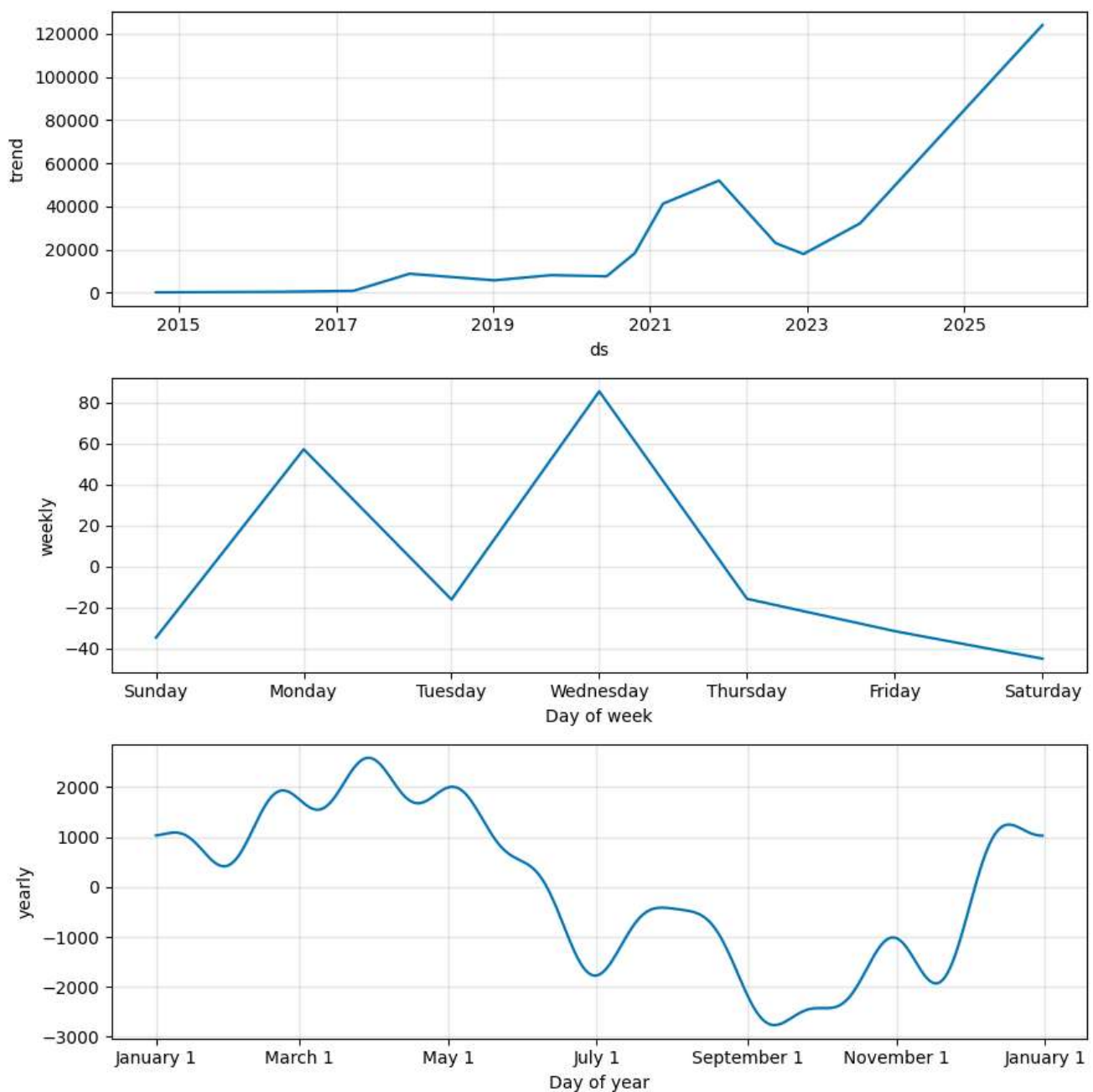
RMSE : 2557.7912438239177

9. Prophet Model

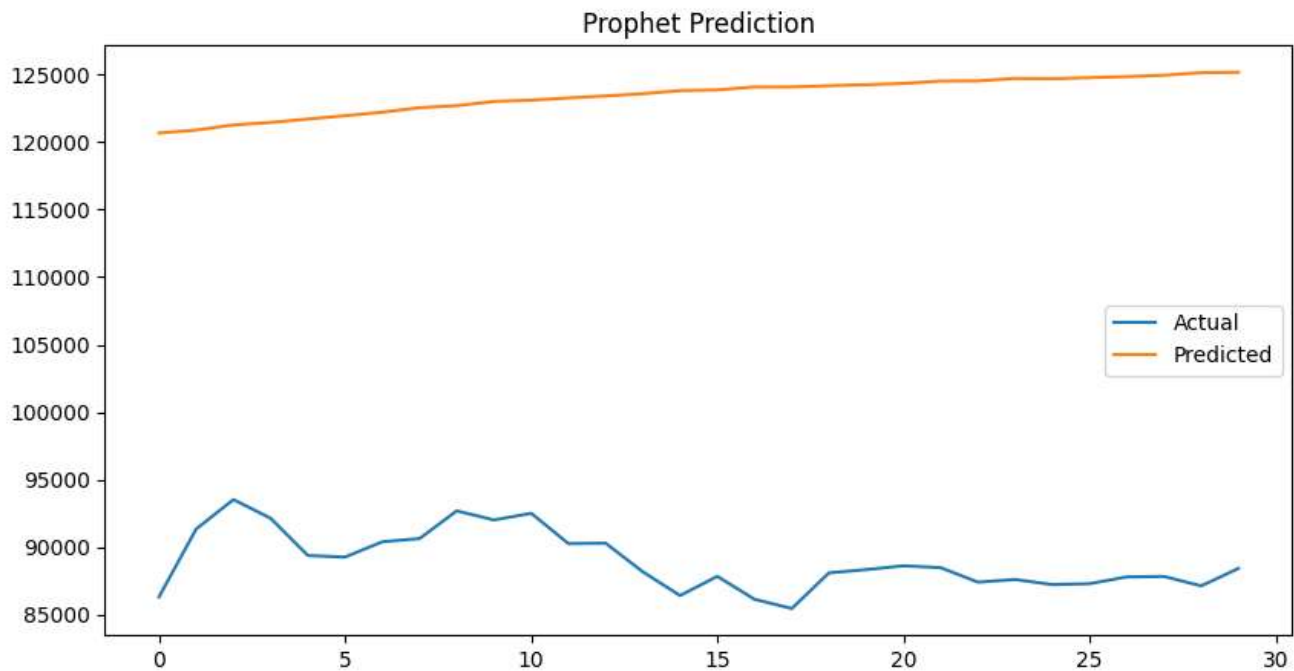
- The Date and Close columns were renamed and cleaned.
- The model was trained using all data except the last 30 days.
- A future dataframe was generated for 30 additional days
- The model predicted the closing prices for the testing period.
- Prophet automatically decomposed the series into:
 - weekly seasonality
 - Yearly seasonality
 - Trends

Prophet handles missing values and trend changes well and provides interpretable forecasts.

Visualization 7: Prophet Forecast Plot



Visualization 8: Prophet Actual vs Predicted Prices



Visualization 9: Prophet Error Metrics

MSE : 1198472580.159996
RMSE : 34618.962725073034

10. LSTM Model

10.1 Introduction to LSTM

LSTM (Long Short-Term Memory) is a specialized recurrent neural network designed for sequential data.

It can retain important information over long periods, making it highly effective for time series forecasting.

10.2 Data Scaling

Neural networks perform better when data is normalized. MinMaxScaler transformed prices to a range between 0 and 1.

10.3 Sequence Creation

A window size of 10 days was used.

Each input sample consisted of 10 consecutive closing prices, and the output was the next day's price.

10.4 Train-Test Split

80% of the sequences were used for training, and 20% were used for testing.

10.5 Model Architecture

The network contained:

- One LSTM layer with 50 units.
- One Dense output layer.

10.6 Compilation

- Optimizer: Adam
- Loss Function: Mean Squared Error

10.7 Training

The model was trained for 10 epochs with a batch size of 16.

10.8 Prediction

Predictions were generated and converted back to the original price scale.

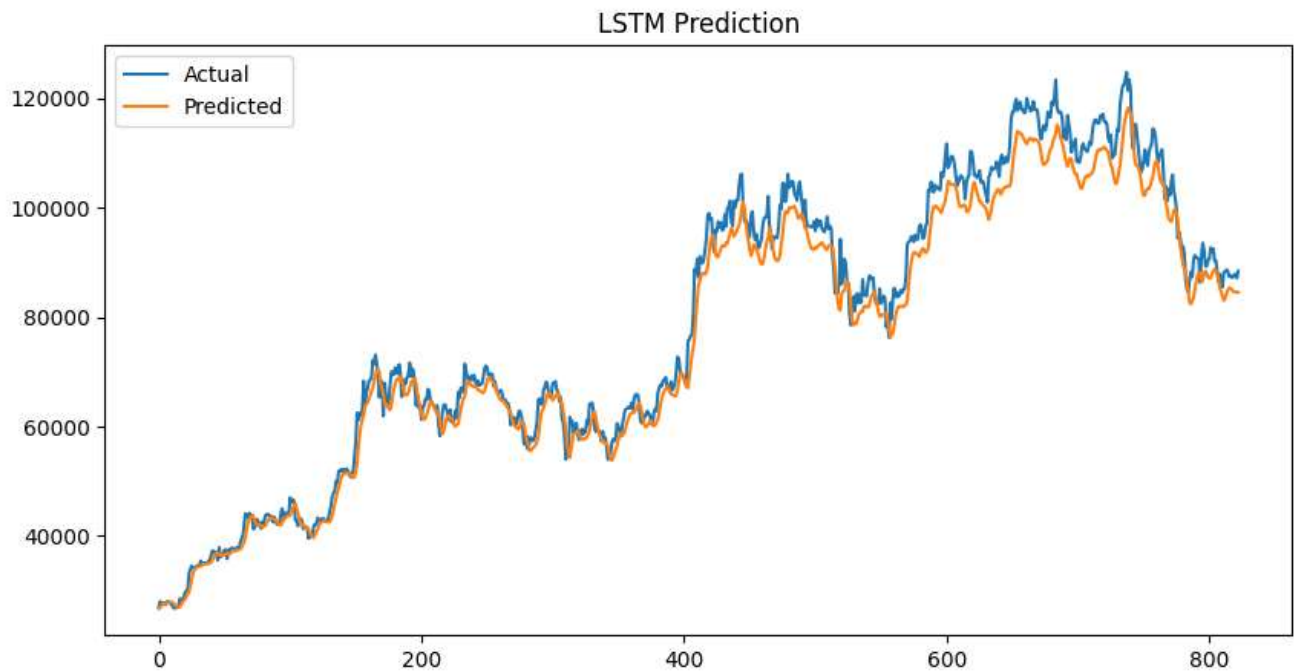
10.9 Interpretation

LSTM learns complex and non-linear patterns and often provides the most accurate results.

Visualization 10: LSTM Training Progress (Loss Curve)



Visualization 11: LSTM Actual vs Predicted Prices



Visualization 12: LSTM Error Metrics

MSE: 15387300.462774975

RMSE: 3922.6649694786547

13. Results and Discussion

ARIMA

- *Good baseline model.*
- *Suitable for linear trends.*
- *Performance decreases with high volatility.*

Prophet

- *Provides interpretable components.*
- *Handles trend changes effectively.*
- *Performs better than ARIMA in many cases.*

LSTM

- *Best at capturing non-linear patterns.*
- *Typically provides the lowest RMSE.*
- *Most suitable for cryptocurrency forecasting.*

Overall Observation

Bitcoin prices are highly non-linear and volatile. Deep learning approaches such as LSTM are generally better suited for this type of data.

14. Conclusion

This project successfully developed a cryptocurrency price prediction system using ARIMA, Prophet, and LSTM.

Key Highlights of the Project

- **Historical Bitcoin data** was downloaded and cleaned successfully.
- **Three forecasting models** were implemented and compared.
- **MSE and RMSE** were used for performance evaluation.
- **LSTM** delivered the most accurate predictions.
- **Prophet** provided highly interpretable trend and seasonality analysis.
- **ARIMA** served as a strong statistical baseline.
- **Streamlit dashboard** enabled interactive exploration of real-time Bitcoin data.
- **19 visualizations** were included to explain the full workflow.

Final Conclusion

The project demonstrates that machine learning and deep learning techniques can effectively forecast cryptocurrency prices. Among all models, LSTM performed best because it captured the complex and non-linear behavior of Bitcoin prices more accurately than ARIMA and Prophet.

15. Future Scope

The project can be improved by:

- *Using additional features such as volume, technical indicators, and sentiment data.*
 - *Hyperparameter tuning.*
 - *Training deeper neural networks.*
 - *Deploying the model in a web application.*
 - *Predicting multiple cryptocurrencies.*
-

Streamlit Dashboard Application

Introduction to the Dashboard

In addition to building forecasting models, an interactive web dashboard was developed using [Streamlit](https://streamlit.io) <https://streamlit.io>. The dashboard allows users to explore Bitcoin price data visually and view important metrics in real time.

The application automatically downloads the latest Bitcoin data from [Yahoo Finance](https://finance.yahoo.com) <https://finance.yahoo.com> using the `yfinance` library and presents it in an easy-to-use interface.

Purpose of the Dashboard

The dashboard was created to:

- *Display current Bitcoin price information.*
- *Show historical price trends.*
- *Analyze volatility using rolling statistics.*
- *Provide descriptive statistics.*
- *Allow users to view the dataset interactively.*

Technologies Used

The dashboard was developed using:

- Python
- Streamlit
- Plotly Express
- yfinance
- NumPy
- Pandas

Dashboard Layout

The application contains the following sections:

1. *Title Header*
2. *Sidebar Controls*
3. *Market Sentiment Indicator*
4. *Volatility Indicator*
5. *Key Performance Metrics (KPIs)*
6. *Bitcoin Price Trend Chart*
7. *Volatility Analysis Chart*
8. *Dataset Viewer*
9. *Statistical Summary*
10. *Footer*

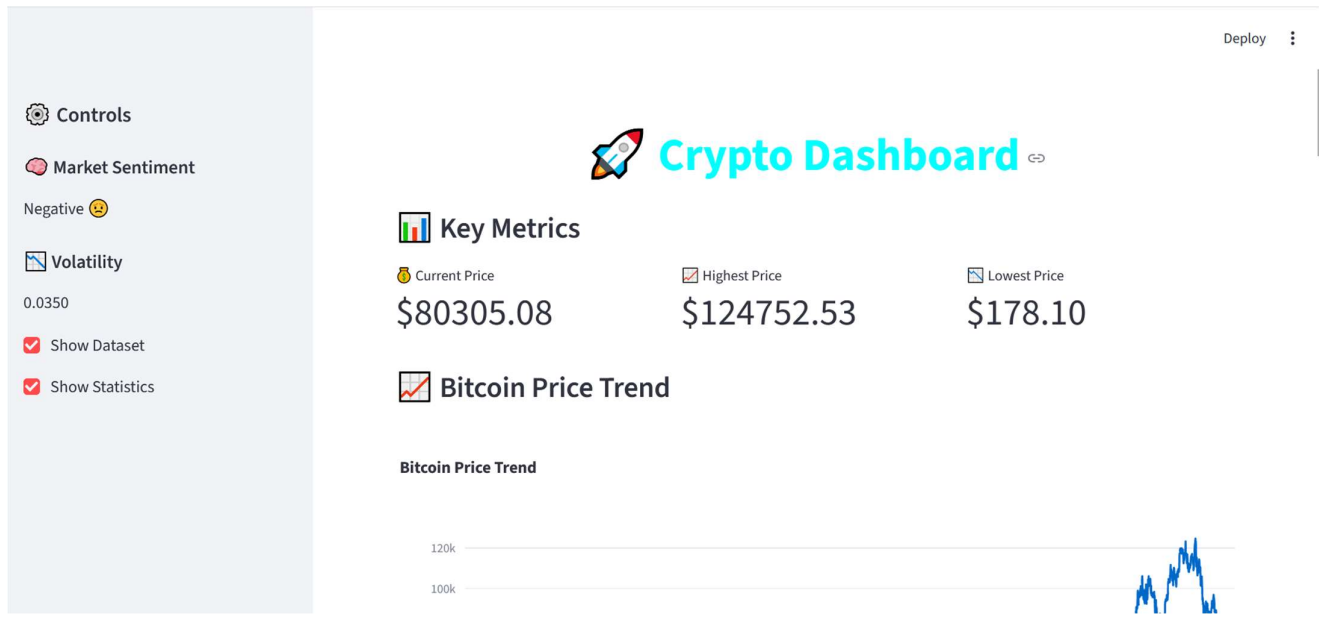
Header Section

A centered title named "Crypto Dashboard" is displayed at the top of the application.

Purpose

- *Provides a professional look.*
- *Clearly identifies the dashboard.*

Visualization 13: Dashboard Home Screen



Data Loading

The dashboard downloads Bitcoin data dynamically using the BTC-USD ticker.

Purpose

- Ensures the dashboard always displays the latest market data.
- Eliminates the need to manually update files.

Data Cleaning

The downloaded data is cleaned by:

- Flattening MultiIndex columns.
- Resetting the index.
- Converting Date to datetime format.
- Converting Close to numeric.
- Removing missing values.
- Sorting records by date.

This ensures that all visualizations are accurate.

Sidebar Controls

The sidebar contains interactive options.

Market Sentiment

A simulated sentiment indicator randomly displays one of the following:

- Positive 😊
- Negative 😞

- Neutral 😐

Volatility Indicator

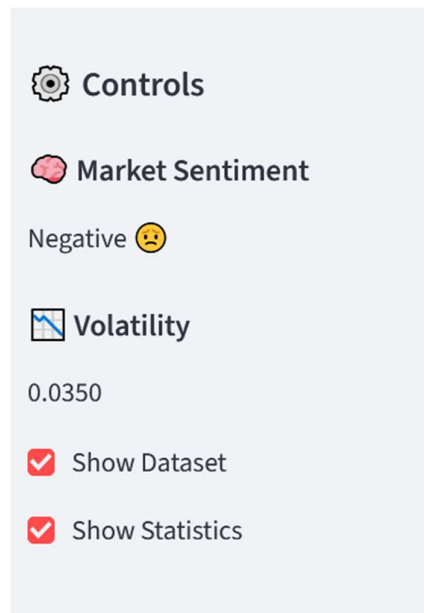
Volatility is calculated using the standard deviation of percentage returns.

Checkboxes

Users can choose to:

- Show the dataset.
- Show descriptive statistics.

Visualization 14: Sidebar Controls



Key Metrics Section

Three KPI cards are displayed.

Current Price

Shows the latest closing price.

Highest Price

Displays the maximum historical closing price.

Lowest Price

Displays the minimum historical closing price.

These metrics help users quickly understand the overall market position.

Visualization 15: KPI Metrics Cards



Key Metrics



Current Price

\$80270.59



Highest Price

\$124752.53



Lowest Price

\$178.10

Bitcoin Price Trend Chart

An interactive line chart displays Bitcoin closing prices over time.

Features

- Zooming
- Hover tooltips
- Pan controls
- Full-screen mode

Insight

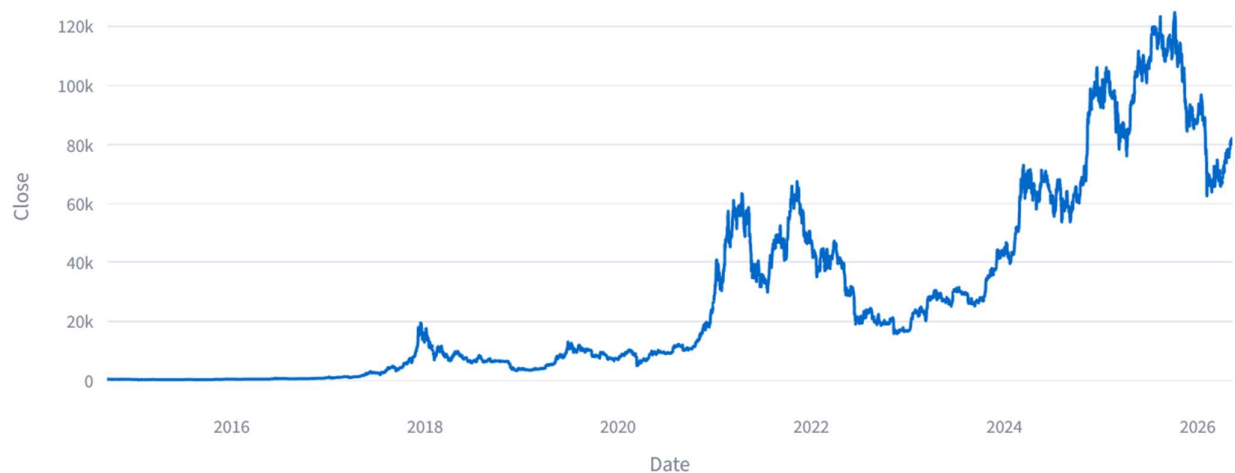
This chart clearly shows long-term growth and periods of market volatility.

Visualization 16: Bitcoin Price Trend Chart



Bitcoin Price Trend

Bitcoin Price Trend



Volatility Analysis Chart

Rolling Mean and Rolling Standard Deviation are calculated using a 20-day window.

Rolling Mean

Represents the smoothed average price.

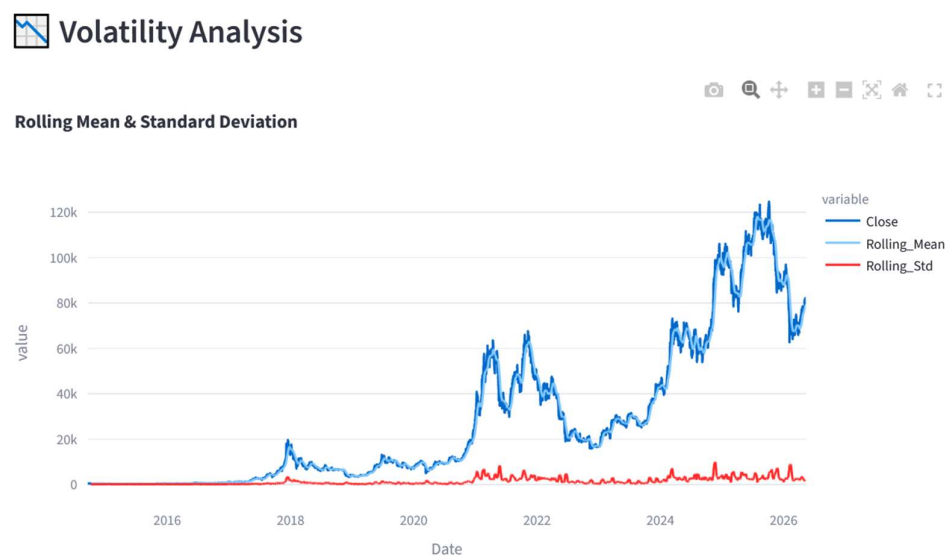
Rolling Standard Deviation

Measures market volatility.

Insight

Higher standard deviation indicates greater uncertainty in price movements.

Visualization 17: Rolling Mean and Standard Deviation Chart



Dataset Viewer

When the "Show Dataset" checkbox is selected, the last few rows of the dataset are displayed in a table.

Purpose

- Enables users to inspect recent data.
 - Improves transparency.
-

Visualization 18: Dataset Table

Dataset

	Date	Close	High	Low	Open	Volume	Rolling_Mean	Rolling_Std
4251	2026-05-08 00:00:00	80186.7656	80447.2656	79205.5156	80009.625	33789351540	77991.5023	1913.6077
4252	2026-05-09 00:00:00	80664.3672	81030.0625	80119.9297	80187.7422	18102086996	78331.9031	1736.6524
4253	2026-05-10 00:00:00	82138.9297	82430.1719	80274.1328	80665.6094	26965971077	78645.2234	1832.2374
4254	2026-05-11 00:00:00	81728.2969	82326.2344	80451.4219	82139.0469	32409774919	78913.9996	1872.094
4255	2026-05-12 00:00:00	80305.0781	81721.4141	80188.1016	81721.4141	32628260864	79019.0984	1889.0097

Statistical Summary

Visualization 19: Statistics Table

Statistics

	Date	Close	High	Low	Open	Volume	Rolling_Mean	Rolling
count	4256	4256	4256	4256	4256	4256	4237	
mean	2020-07-14 12:00:00	28379.3668	28926.2163	27764.1632	28361.6189	22230849937.9037	28326.5534	1346.
min	2014-09-17 00:00:00	178.103	211.731	171.51	176.897	5914570	226.3832	2.
25%	2017-08-15 18:00:00	3410.1938	3476.1843	3293.6563	3394.4874	1562317536	3553.9924	128.
50%	2020-07-14 12:00:00	11462.8735	11758.6367	11182.5708	11430.4497	17880725007	11264.4569	681.
75%	2023-06-13 06:00:00	45563.252	46800.2041	44168.9424	45546.8076	34380861563.5	44802.365	2230.
max	2026-05-12 00:00:00	124752.5313	126198.0703	123196.0469	124752.1406	350967941479	118311.752	9817.
std	None	32349.7954	32891.2849	31757.0883	32343.4901	22964739405.6366	32239.5128	1581.

 Advanced Dashboard using Streamlit + Plotly + yfinance

Footer

A footer message is displayed at the bottom of the dashboard indicating that the application was built using Streamlit, Plotly, and yfinance.

How to Run the Application

To launch the dashboard, run the following command in the terminal:

```
streamlit run app.py
```

The application will open automatically in the browser.

Dashboard Conclusion

The Streamlit dashboard transforms the forecasting project into an interactive application. It allows users to visualize Bitcoin market behavior and explore historical data without writing code.

Applications

This project can be used for:

- *Investment analysis*
- *Trading strategies*
- *Risk assessment*
- *Educational purposes*
- *Research in financial forecasting*

Limitations

- *Uses only historical closing prices.*
- *Does not incorporate market news or sentiment.*
- *Performance depends on the chosen hyperparameters.*
- *Cryptocurrency markets are inherently unpredictable.*

References

1. *Yahoo Finance.*
2. *yfinance Python library documentation.*
3. *Statsmodels documentation.*
4. *Prophet documentation.*
5. *TensorFlow and Keras documentation.*
6. *Scikit-learn documentation.*

Future scope of the dashboard

The Streamlit dashboard can be enhanced by adding real-time price updates, support for multiple cryptocurrencies, and integration of ARIMA, Prophet, and LSTM prediction results. Additional features such as technical indicators, sentiment analysis, buy/sell signals, and price alerts can further improve its usefulness. The dashboard can also be deployed online to provide users with access from any device.